Easy Regex Reference (Python)

Python re quick reference — patterns, meaning, example, and sample matches.

PATTERN : sample

MEANING : Literal text 'sample'.

EXAMPLE : This is a sample line; another sample appears here.

MATCHES : ['sample', 'sample']

PATTERN : .

MEANING : Any character except newline (use DOTALL/S to include newlines).

EXAMPLE : A.
B
MATCHES : ['A', '.', 'B']

PATTERN : \d \D \w \W \s \S

MEANING : Shorthands: digit / non-digit / word / non-word / whitespace / non-whitespace.

EXAMPLE : Id=A9 Val: 42
Done
MATCHES : (syntax reference only)

PATTERN : [A-Z]{2}\d{3}

MEANING : Two capitals followed by three digits.

EXAMPLE : Codes: AB123, aa123, XY999.

MATCHES : ['AB123', 'XY999']

PATTERN : [^aeiou]

MEANING : Negated class: any character that is NOT a vowel (lowercase).

EXAMPLE : queue

MATCHES : ['q']

PATTERN : ^The

MEANING : Start-of-string/line anchor (with MULTILINE). Matches 'The' at start.

EXAMPLE : The first line
Another line
The last one
MATCHES : ['The', 'The']

PATTERN : end$

Easy Regex Reference (Python) (cont.)

MEANING : End-of-string/line anchor (with MULTILINE).

EXAMPLE : Reach the end
not here end
MATCHES : ['end', 'end']

PATTERN : \bcat\b

MEANING : Word boundary: whole word 'cat' (won't match 'concatenate').

EXAMPLE : cat scat concatenate cat.

MATCHES : ['cat', 'cat']

PATTERN : a+ a* a? a{2,4} a+? a*? a??

MEANING : Greedy, optional, and lazy quantifiers. (Shown as patterns only.)

EXAMPLE : aaaa

MATCHES : (syntax reference only)

PATTERN : (cat|dog)

MEANING : Grouping + alternation (OR). Captures either 'cat' or 'dog'.

EXAMPLE : my cat and your dog chase a catdog

MATCHES : ['cat', 'dog', 'cat', 'dog']

PATTERN : (?:cat|dog)

MEANING : Non-capturing group.

EXAMPLE : cat dog catalog

MATCHES : ['cat', 'dog', 'cat']

PATTERN : (?P<area>\d{3})-(?P<ex>\d{3})-(?P<line>\d{4})

MEANING : Named groups.

EXAMPLE : Phones: 123-456-7890 and 555-0100 (bad).

MATCHES : ['123-456-7890']

PATTERN : (\w+)\s+\1

MEANING : Backreference: repeated word.

EXAMPLE : this is is a test test of repeats

MATCHES : ['is is', 'test test']

PATTERN : (?<=USD)\s*\d+(?:\.\d{2})?

MEANING : Positive lookbehind: numbers preceded by 'USD'.

EXAMPLE : USD 19.99, EUR 29.99, USD200

MATCHES : [' 19.99', '200']

PATTERN : \b\w+(?=\.)

MEANING : Positive lookahead: word before a dot (filename base).

EXAMPLE : report.pdf archive.tar.gz notes

MATCHES : ['report', 'archive', 'tar']

PATTERN : \b(?!cat)\w+\b

MEANING : Negative lookahead: words that are NOT 'cat'.

EXAMPLE : cat bat mat cat

MATCHES : ['bat', 'mat']

PATTERN : (?i)hello

MEANING : Inline flag: case-insensitive 'hello'.

EXAMPLE : Hello hELLo hey

MATCHES : ['Hello', 'hELLo']

PATTERN : (?m)^item

MEANING : Inline multiline: ^ and $ now work per line.

EXAMPLE : item one
not it
item two

MATCHES : ['item', 'item']

PATTERN : (?s)a.*z

MEANING : DOTALL: dot matches newlines too.

EXAMPLE : a...
...
z

MATCHES : ['a...\n...\nz']

Easy Regex Reference (Python) (cont.)

PATTERN : \b\d{3}-\d{3}-\d{4}\b

MEANING : US phone (dashed).

EXAMPLE : Call 123-456-7890 or 987-654-3210 today.

MATCHES : ['123-456-7890', '987-654-3210']

PATTERN : \b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b

MEANING : Email (good heuristic; not RFC-complete).

EXAMPLE : Send to a.b+c@example.co.uk or bad@mail..com

MATCHES : ['a.b+c@example.co.uk', 'bad@mail..com']

PATTERN : \b\d{4}-\d{2}-\d{2}\b

MEANING : Date YYYY-MM-DD.

EXAMPLE : Due: 2023-03-11, also 11/03/2023

MATCHES : ['2023-03-11']

PATTERN : #[A-Fa-f0-9]{6}\b

MEANING : Hex color #RRGGBB.

EXAMPLE : Primary: #1F77B4, invalid: #ZZZZZZ

MATCHES : ['#1F77B4']

PATTERN : \b\d{1,3}(?:\.\d{1,3}){3}\b

MEANING : IPv4 (not range-checked).

EXAMPLE : Gateway 192.168.1.1; text 999.999.999.999

MATCHES : ['192.168.1.1', '999.999.999.999']

PATTERN : \bhttps?://[^\s/$.?#].[^\s]*

MEANING : Basic URL (heuristic).

EXAMPLE : Visit https://example.com/docs?id=1 or http://bad url

MATCHES : ['https://example.com/docs?id=1', 'http://bad']

PATTERN : \b[a-f0-9]{8}-[a-f0-9]{4}-4[a-f0-9]{3}-[89ab][a-f0-9]{3}-[a-f0-9]{12}\b

MEANING : UUID v4.

Easy Regex Reference (Python) (cont.)

EXAMPLE : id=123e4567-e89b-12d3-a456-426614174000

MATCHES : []

PATTERN : \b[A-Z]{2}-\d{4}-\d{4}\b

MEANING : Order code AB-1234-5678.

EXAMPLE : Orders: AB-1234-5678, XY-0000-0001

MATCHES : ['AB-1234-5678', 'XY-0000-0001']

PATTERN : \b[\w.\-]+\.(?:pdf|docx|xlsx|pptx)\b

MEANING : Filenames with allowed extensions.

EXAMPLE : Files: Report Final.pdf, notes.docx, image.png

MATCHES : ['Report Final.pdf', 'notes.docx']

PATTERN : $?\d+(?:\.\d{2})?

MEANING : Price: optional $, integers or .cc.

EXAMPLE : Items: $19.99, 5, 12.00

MATCHES : ['$19.99', '5', '12.00']

PATTERN : re.sub(r'(\d{3})-(\d{3})-(\d{4})', r'\1-\2-XXXX', text)

MEANING : Substitution: mask last 4 digits using backreferences.

EXAMPLE : Call 123-456-7890

MATCHES : (syntax reference only)
